

PROACTI-SHIELD: A Predictive Confidence-Driven Border Intrusion Intelligence System Based on Real-Time Multi-Sensor Event Correlation

Code :

final code

```
#!/usr/bin/env python3
```

```
import os, csv, time, threading, glob
```

```
from datetime import datetime, date
```

```
from flask import Flask, jsonify, render_template_string, request
```

```
import RPi.GPIO as GPIO
```

```
# ===== PIN CONFIG (BCM) =====
```

```
# Inputs (digital)
```

```
SOUND_PIN = 17
```

```
VIB_PIN = 27
```

```
PIR_PIN = 22
```

```
# Outputs
```

```
BUZZER = 18
```

```
LED = 23
```

```
# ===== AGENT CONFIG =====
```

```
THRESHOLD_ALERT = 70
```

```
THRESHOLD_WARN = 55
```

```
# Weights (0/1 digital inputs)
```

```
W_VIB, W_PIR, W_TEMP, W_SOUND = 40, 30, 20, 10
```

```
# Temperature rule (DS18B20)
```

```
TEMP_ON_C = 30.0 # temp_flag = 1 if temp >= TEMP_ON_C
```

```
# Agent behavior
```

```
MEM_WINDOW_SEC = 10
```

```
REPEAT_FOR_ALERT = 4
```

```
ALARM_COOLDOWN_SEC = 10
```

```
BEEP_ON_SEC = 0.15
```

```
BEEP_OFF_SEC = 0.10
```

```
BEEP_COUNT = 3
```

```
LOOP_SLEEP_SEC = 1
```

```

# ===== GPIO SETUP =====
GPIO.setmode(GPIO.BCM)

GPIO.setup(PIR_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(SOUND_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(VIB_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

GPIO.setup(BUZZER, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)
GPIO.output(BUZZER, 0)
GPIO.output(LED, 0)

# ===== DS18B20 SETUP =====
USE_DS18B20 = True

device_file = None
if USE_DS18B20:
    os.system("modprobe w1-gpio")
    os.system("modprobe w1-therm")
    base_dir = "/sys/bus/w1/devices/"
    device_folders = glob.glob(base_dir + "28-*")
    device_file = device_folders[0] + "/w1_slave" if device_folders else None

def read_ds18b20():
    if not device_file:
        return None
    try:
        with open(device_file, "r") as f:
            lines = f.readlines()
            if not lines or lines[0].strip()[-3:] != "YES":
                return None
            return float(lines[1].split("t=")[-1]) / 1000.0
    except:
        return None

# ===== LOG SETUP =====
os.makedirs("logs", exist_ok=True)

def log_file_path():
    return os.path.join("logs", f"{date.today().isoformat()}.csv")

```

```

def append_log(row):
    path = log_file_path()
    new_file = not os.path.exists(path)
    with open(path, "a", newline="") as f:
        w = csv.writer(f)
        if new_file:
            w.writerow([
                "timestamp","vibration","pir","sound","temp_c","temp_flag",
                "total","ai_level","ai_action","ai_message"
            ])
        w.writerow(row)

```

```

def read_today_rows():
    path = log_file_path()
    if not os.path.exists(path):
        return []
    with open(path, "r") as f:
        return list(csv.DictReader(f))

```

```

def today_alert_count():
    rows = read_today_rows()
    return sum(1 for r in rows if r.get("ai_level") == "ALERT")

```

```

def last_alert_time():
    rows = read_today_rows()
    for r in reversed(rows):
        if r.get("ai_level") == "ALERT":
            return r.get("timestamp")
    return None

```

```

def max_total_today():
    rows = read_today_rows()
    mx = 0
    for r in rows:
        try:
            val = int(float(r.get("total", 0)))
            if val > mx:
                mx = val
        except:
            pass
    return mx

```

```

# ===== AGENTIC AI (MEMORY + RULES) =====
events = [] # list of (ts, vib, pir, sound, temp_flag, total)

def prune_events(now_ts):
    global events
    cutoff = now_ts - MEM_WINDOW_SEC
    events = [e for e in events if e[0] >= cutoff]

def count_hits(index, now_ts):
    prune_events(now_ts)
    return sum(1 for e in events if e[index] == 1)

def agent_score(vib, pir, temp_flag, sound):
    return vib*W_VIB + pir*W_PIR + temp_flag*W_TEMP + sound*W_SOUND

def agent_decide(vib, pir, temp_flag, sound, total, now_ts):
    events.append((now_ts, vib, pir, sound, temp_flag, total))

    vib_hits = count_hits(1, now_ts)
    pir_hits = count_hits(2, now_ts)
    snd_hits = count_hits(3, now_ts)
    tmp_hits = count_hits(4, now_ts)

    if total >= THRESHOLD_ALERT:
        ai_level = "ALERT"
        ai_action = "alarm"
        ai_message = "High score. Human likely."
    elif total >= THRESHOLD_WARN:
        ai_level = "WARNING"
        ai_action = "log"
        ai_message = "Suspicious activity. Monitoring."
    else:
        ai_level = "NORMAL"
        ai_action = "none"
        ai_message = "Stable. No significant activity."

    repeat_confirm = (
        (pir_hits >= 2) or
        (snd_hits >= REPEAT_FOR_ALERT and vib_hits >= REPEAT_FOR_ALERT) or
        (tmp_hits >= 2)

```

)

```
if ai_level == "ALERT" and repeat_confirm:
    ai_action = "alarm"
    ai_message = f"Confirmed by repeats (PIR:{pir_hits}, Sound:{snd_hits}, Vib:{vib_hits},
TempFlag:{tmp_hits})."
elif ai_level == "ALERT" and not repeat_confirm:
    ai_action = "beep"
    ai_message = "Alert from score, but not enough repeats yet. Beep only."

hits = {"vib": vib_hits, "pir": pir_hits, "sound": snd_hits, "temp": tmp_hits}
return ai_level, ai_action, ai_message, hits
```

```
# ===== ACTION CONTROL =====
last_alarm_time = 0.0
```

```
def beep():
    for _ in range(BEEP_COUNT):
        GPIO.output(BUZZER, 1)
        time.sleep(BEEP_ON_SEC)
        GPIO.output(BUZZER, 0)
        time.sleep(BEEP_OFF_SEC)
```

```
def do_actions(ai_level, ai_action, now_ts):
    global last_alarm_time
```

```
    if ai_level == "ALERT":
        GPIO.output(LED, 1)
    elif ai_level == "WARNING":
        GPIO.output(LED, 1)
    else:
        GPIO.output(LED, 0)
```

```
    if ai_action in ("alarm", "beep"):
        if (now_ts - last_alarm_time) >= ALARM_COOLDOWN_SEC:
            last_alarm_time = now_ts
            beep()
    else:
        GPIO.output(BUZZER, 0)
```

```
# ===== FLASK APP + LIVE DATA =====
```

```
app = Flask(__name__)
lock = threading.Lock()
```

```
latest = {
    "timestamp": "",
    "vibration": 0,
    "pir": 0,
    "sound": 0,
    "temp_c": None,
    "temp_flag": 0,
    "total": 0,
    "ai_level": "NORMAL",
    "ai_action": "none",
    "ai_message": "",
    "hits": {"vib":0, "pir":0, "sound":0, "temp":0}
}
```

```
# ===== DASHBOARD HTML =====
```

```
HTML = r"""
```

```
<!doctype html>
```

```
<html>
```

```
<head>
```

```
<meta charset="utf-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

```
<title>Agentic AI Dashboard</title>
```

```
<style>
```

```
:root{
```

```
--bg:#0b1020;
```

```
--panel:#101a33;
```

```
--text:#e9eefc;
```

```
--muted:#a9b3d2;
```

```
--border:rgba(255,255,255,.08);
```

```
--shadow: 0 10px 30px rgba(0,0,0,.35);
```

```
--good:#3ddc97;
```

```
--warn:#ffb020;
```

```
--bad:#ff4d6d;
```

```
--blue:#5aa8ff;
```

```
}
```

```
*{ box-sizing:border-box; }
```

```
body{
```

```

margin:0;
font-family: ui-sans-serif, system-ui, Arial;
background: radial-gradient(1200px 600px at 20% -10%, rgba(90,168,255,.25),
transparent 60%),
            radial-gradient(900px 500px at 90% 0%, rgba(255,77,109,.20), transparent 60%),
            var(--bg);
color:var(--text);
}
.wrap{ max-width:1100px; margin:0 auto; padding:24px; }
.topbar{
display:flex; align-items:flex-start; justify-content:space-between; gap:14px;
margin-bottom:16px;
}
.title{ font-size:28px; font-weight:800; margin:0; }
.subtitle{ margin-top:6px; color:var(--muted); font-size:13px; }
.chip{
display:inline-flex; align-items:center; gap:8px;
background:rgba(255,255,255,.06);
border:1px solid var(--border);
padding:10px 12px;
border-radius:999px;
box-shadow: var(--shadow);
min-width: 220px;
justify-content: center;
}
.dot{
width:10px; height:10px; border-radius:50%;
background: var(--good);
box-shadow: 0 0 0 6px rgba(61,220,151,.15);
}
.grid{
display:grid;
grid-template-columns: 1.2fr .8fr;
gap:16px;
}
@media (max-width: 900px){
.grid{ grid-template-columns: 1fr; }
}
.panel{
background: linear-gradient(180deg, rgba(255,255,255,.06), rgba(255,255,255,.03));
border:1px solid var(--border);

```

```

border-radius:20px;
box-shadow: var(--shadow);
overflow:hidden;
}
.panel-h{
padding:16px 18px;
display:flex; align-items:center; justify-content:space-between;
border-bottom:1px solid var(--border);
background: rgba(0,0,0,.10);
}
.pill{
padding:8px 12px;
border-radius:999px;
font-weight:800;
border:1px solid var(--border);
background: rgba(255,255,255,.06);
display:flex; align-items:center; gap:10px;
min-width: 150px;
justify-content: center;
}
.pill .mini-dot{ width:9px; height:9px; border-radius:50%; background: var(--good); }
.pill.normal{ color: var(--good); }
.pill.warning{ color: var(--warn); }
.pill.alert{ color: var(--bad); }
.content{ padding:16px 18px 18px; }
.cards{
display:grid;
grid-template-columns: repeat(3, 1fr);
gap:12px;
}
@media (max-width: 900px){
.cards{ grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 560px){
.cards{ grid-template-columns: 1fr; }
}
.card{
background: linear-gradient(180deg, rgba(255,255,255,.05), rgba(255,255,255,.02));
border:1px solid var(--border);
border-radius:16px;
padding:14px;

```



```

}
.k{
  color:var(--muted);
  font-size:12px;
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:10px;
}
.v{ margin-top:10px; font-size:28px; font-weight:900; }
.subv{ margin-top:2px; font-size:12px; color:var(--muted); }
.meter{
  margin-top:12px;
  height:12px;
  border-radius:999px;
  background: rgba(255,255,255,.08);
  border: 1px solid var(--border);
  overflow:hidden;
}
.meter > div{
  height:100%;
  width:0%;
  background: linear-gradient(90deg, var(--good), var(--warn), var(--bad));
  border-radius:999px;
  transition: width .25s ease;
}
.ai-box, .chat-box{
  margin-top:12px;
  padding:12px 14px;
  border-radius:16px;
  border:1px solid var(--border);
  background: rgba(0,0,0,.14);
}
.ai-box .t, .chat-box .t{
  color:var(--muted);
  font-size:12px;
  margin-bottom:6px;
}
.ai-box .m{ font-size:14px; font-weight:700; }
.chat-row{
  display:flex;

```

```

gap:10px;
flex-wrap:wrap;
margin-top:8px;
}
.chat-row input{
flex:1;
min-width:240px;
padding:10px 12px;
border-radius:12px;
border:1px solid var(--border);
background:rgba(255,255,255,.06);
color:white;
outline:none;
}
.chat-row button{
padding:10px 16px;
border:none;
border-radius:12px;
background:var(--blue);
color:white;
cursor:pointer;
font-weight:700;
}
.chat-reply{
margin-top:10px;
padding:10px 12px;
border-radius:12px;
background:rgba(255,255,255,.05);
color:#fff;
min-height:44px;
}
.chart-wrap{
margin-top:14px;
padding:14px;
border-radius:16px;
border:1px solid var(--border);
background: rgba(0,0,0,.14);
}
canvas{ width:100%; height:220px; display:block; }
.kbd{
font-family: ui-monospace, monospace;

```

```

border:1px solid var(--border);
border-radius:10px;
padding:4px 8px;
background: rgba(255,255,255,.06);
color: var(--text);
}
</style>
</head>

<body>
<div class="wrap">
<div class="topbar">
<div>
<h1 class="title">Agentic AI Multi-Sensor Dashboard</h1>
<div class="subtitle">Real-time monitoring • Rules + memory + actions (buzzer/LED)
• Logging enabled</div>
</div>

<div class="chip">
<div class="dot" id="liveDot"></div>
<div>
<div><b id="ts">--</b></div>
<span>Last update</span>
</div>
</div>
</div>

<div class="grid">
<div class="panel">
<div class="panel-h">
<div>
<b>Live Sensors</b><br>
<span style="color:var(--muted);font-size:12px;">Vibration • PIR • Sound •
Temperature</span>
</div>
<div class="pill normal" id="pill">
<span class="mini-dot"></span>
<span id="level">NORMAL</span>
</div>
</div>

```

```

<div class="content">
  <div class="cards">
    <div class="card">
      <div class="k"><span>Vibration</span><span class="kbd"
id="hv">0</span></div>
      <div class="v" id="vib">0</div>
      <div class="subv">Hits (last window)</div>
    </div>

    <div class="card">
      <div class="k"><span>PIR</span><span class="kbd" id="hp">0</span></div>
      <div class="v" id="pir">0</div>
      <div class="subv">Hits (last window)</div>
    </div>

    <div class="card">
      <div class="k"><span>Sound</span><span class="kbd" id="hs">0</span></div>
      <div class="v" id="snd">0</div>
      <div class="subv">Hits (last window)</div>
    </div>

    <div class="card">
      <div class="k"><span>Temp (°C)</span><span class="kbd">Flag: <span
id="tmpf">0</span></span></div>
      <div class="v" id="tmpc">NA</div>
      <div class="subv">Threshold flag</div>
    </div>

    <div class="card" style="grid-column: span 2;">
      <div class="k"><span>Total Score</span><span class="kbd"
id="total">0</span></div>
      <div class="meter"><div id="bar"></div></div>
      <div style="margin-top:10px;color:var(--muted);font-size:12px;">
        Warning ≥ <b id="tw"></b> | Alert ≥ <b id="ta"></b> | Window <b id="win"></b>s
      </div>
    </div>

    <div class="ai-box">
      <div class="t">AI Message</div>
      <div class="m" id="msg">--</div>

```

```

</div>

<div class="chat-box">
  <div class="t">Ask AI</div>
  <div class="chat-row">
    <input id="chatInput" type="text" placeholder="Type: today report / current status
/ last alert / alert count / max total">
    <button onclick="sendChat()">Send</button>
  </div>
  <div class="chat-reply" id="chatReply">AI: Ask something...</div>
</div>
</div>
</div>

```

```

<div class="panel">
  <div class="panel-h">
    <div>
      <b>Score Trend</b><br>
      <span style="color:var(--muted);font-size:12px;">Last ~60 samples</span>
    </div>
    <div class="kbd">1 Hz</div>
  </div>
  <div class="content">
    <div class="chart-wrap">
      <canvas id="c"></canvas>
    </div>
  </div>
</div>
</div>
</div>

```

```

<script>
const TH_WARN = %TH_WARN%;
const TH_ALERT = %TH_ALERT%;
const WIN = %WIN%;

document.getElementById("tw").innerText = TH_WARN;
document.getElementById("ta").innerText = TH_ALERT;
document.getElementById("win").innerText = WIN;

const canvas = document.getElementById("c");

```

```

const ctx = canvas.getContext("2d");

function resizeCanvas(){
  const rect = canvas.getBoundingClientRect();
  canvas.width = Math.floor(rect.width * devicePixelRatio);
  canvas.height = Math.floor(220 * devicePixelRatio);
}
window.addEventListener("resize", resizeCanvas);
resizeCanvas();

let series = [];
const MAX = 60;

function yFor(v){
  return canvas.height - (v/100)*canvas.height;
}

function draw(){
  const w = canvas.width, h = canvas.height;
  ctx.clearRect(0,0,w,h);

  ctx.strokeStyle = "rgba(255,255,255,.20)";
  for(let i=1;i<=4;i++){
    const y = (h/5)*i;
    ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(w,y); ctx.stroke();
  }

  ctx.strokeStyle = "rgba(255,176,32,.55)";
  ctx.beginPath(); ctx.moveTo(0, yFor(TH_WARN)); ctx.lineTo(w, yFor(TH_WARN));
  ctx.stroke();

  ctx.strokeStyle = "rgba(255,77,109,.55)";
  ctx.beginPath(); ctx.moveTo(0, yFor(TH_ALERT)); ctx.lineTo(w, yFor(TH_ALERT));
  ctx.stroke();

  if(series.length < 2) return;

  ctx.strokeStyle = "rgba(90,168,255,.95)";
  ctx.lineWidth = Math.max(2*devicePixelRatio, 2);

  const step = w / (MAX-1);

```

```

ctx.beginPath();
for(let i=0;i<series.length;i++){
  const x = i*step;
  const y = yFor(series[i]);
  if(i===0) ctx.moveTo(x,y); else ctx.lineTo(x,y);
}
ctx.stroke();
}

```

```

function setLevel(level){
  const pill = document.getElementById("pill");
  pill.classList.remove("normal","warning","alert");
  if(level === "ALERT") pill.classList.add("alert");
  else if(level === "WARNING") pill.classList.add("warning");
  else pill.classList.add("normal");
  document.getElementById("level").innerText = level;
}

```

```

async function refresh(){
  try{
    const r = await fetch("/data", {cache:"no-store"});
    const d = await r.json();

    document.getElementById("ts").innerText = d.timestamp || "--";
    document.getElementById("vib").innerText = d.vibration;
    document.getElementById("pir").innerText = d.pir;
    document.getElementById("snd").innerText = d.sound;
    document.getElementById("tmpc").innerText = (d.temp_c === null) ? "NA" : d.temp_c;
    document.getElementById("tmpf").innerText = d.temp_flag;
    document.getElementById("total").innerText = d.total;
    document.getElementById("msg").innerText = d.ai_message || "--";

    document.getElementById("hv").innerText = d.hits?.vib ?? 0;
    document.getElementById("hp").innerText = d.hits?.pir ?? 0;
    document.getElementById("hs").innerText = d.hits?.sound ?? 0;

    setLevel(d.ai_level || "NORMAL");

    const pct = Math.max(0, Math.min(100, d.total));
    document.getElementById("bar").style.width = pct + "%";
  }
}

```

```

    series.push(pct);
    if(series.length > MAX) series.shift();
    draw();
  }catch(e){}
}

async function sendChat(){
  const msg = document.getElementById("chatInput").value.trim();
  if(!msg){
    document.getElementById("chatReply").innerText = "AI: Please type a question.";
    return;
  }

  const r = await fetch("/chat", {
    method: "POST",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify({message: msg})
  });

  const d = await r.json();
  document.getElementById("chatReply").innerText = "AI: " + d.reply;
}

document.getElementById("chatInput").addEventListener("keydown", function(e){
  if(e.key === "Enter"){
    sendChat();
  }
});

setInterval(refresh, 1000);
refresh();
</script>
</body>
</html>
""""

```

```

HTML = HTML.replace("%TH_WARN%",
str(THRESHOLD_WARN)).replace("%TH_ALERT%",
str(THRESHOLD_ALERT)).replace("%WIN%", str(MEM_WINDOW_SEC))

```

```

@app.route("/")

```



```

def home():
    return render_template_string(HTML)

@app.route("/data")
def data():
    with lock:
        return jsonify(latest)

# ===== AI CHAT ROUTE =====
@app.route("/chat", methods=["POST"])
def chat():
    q = request.json.get("message", "").lower().strip()

    with lock:
        L = latest.copy()

    if "current" in q or "status" in q or "condition" in q:
        return jsonify({
            "reply": f"Current level is {L['ai_level']}. Total score is {L['total']}. Action is {L['ai_action']}."
        })

    if "today report" in q or ("today" in q and "report" in q):
        alerts = today_alert_count()
        mx = max_total_today()
        la = last_alert_time()
        if la:
            return jsonify({
                "reply": f"Today report: alert count is {alerts}, maximum score is {mx}, last alert time is {la}."
            })
        else:
            return jsonify({
                "reply": f"Today report: alert count is {alerts}, maximum score is {mx}, no alert happened today."
            })

    if "last alert" in q:
        la = last_alert_time()
        if la:
            return jsonify({"reply": f"Last alert occurred at {la}."})

```

```

else:
    return jsonify({"reply": "No alerts today."})

if "alert count" in q or q == "alerts":
    return jsonify({"reply": f"Today alert count is {today_alert_count()}."})

if "max total" in q or "maximum" in q:
    return jsonify({"reply": f"Maximum total score today is {max_total_today()}."})

if "temperature" in q or "temp" in q:
    tc = L["temp_c"]
    if tc is None:
        return jsonify({"reply": "Temperature sensor data is not available now."})
    return jsonify({"reply": f"Current temperature is {tc} degree Celsius. Temperature flag
is {L['temp_flag']}."})

if "help" in q or q == "":
    return jsonify({
        "reply": "Ask: current status, today report, last alert, alert count, max total,
temperature."
    })

    return jsonify({
        "reply": "I did not understand. Try: current status, today report, last alert, alert count,
max total, temperature."
    })

# ===== SENSOR LOOP =====
def sensor_loop():
    global latest
    while True:
        now_ts = time.time()

        sound = int(GPIO.input(SOUND_PIN))
        vib = int(GPIO.input(VIB_PIN))
        pir = int(GPIO.input(PIR_PIN))

        temp_c = read_ds18b20() if USE_DS18B20 else None
        temp_flag = 1 if (temp_c is not None and temp_c >= TEMP_ON_C) else 0

        total = agent_score(vib, pir, temp_flag, sound)

```

```
    ai_level, ai_action, ai_message, hits = agent_decide(vib, pir, temp_flag, sound, total,
now_ts)
```

```
do_actions(ai_level, ai_action, now_ts)
```

```
ts_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
append_log([
    ts_str, vib, pir, sound,
    round(temp_c, 2) if temp_c is not None else None,
    temp_flag, total, ai_level, ai_action, ai_message
])
```

```
with lock:
```

```
    latest = {
        "timestamp": ts_str,
        "vibration": vib,
        "pir": pir,
        "sound": sound,
        "temp_c": round(temp_c, 2) if temp_c is not None else None,
        "temp_flag": temp_flag,
        "total": total,
        "ai_level": ai_level,
        "ai_action": ai_action,
        "ai_message": ai_message,
        "hits": hits
    }
```

```
time.sleep(LOOP_SLEEP_SEC)
```

```
# ===== MAIN =====
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        t = threading.Thread(target=sensor_loop, daemon=True)
```

```
        t.start()
```

```
        app.run(host="0.0.0.0", port=8000, debug=False)
```

```
    finally:
```

```
        GPIO.output(BUZZER, 0)
```

```
        GPIO.output(LED, 0)
```

```
        GPIO.cleanup()
```